



Agentic Bug Triage & Routing System - SOLUTION PROPOSAL

A unified, AI-powered interface for multi-repository bug management

-
1. [Problem Statement](#)
 - 1.1 [The Core Problem](#)
 - 1.2 [The Four Fragmentation Problems](#)
 - 1.3 [Example Scenario](#)
 2. [Proposed Solution](#)
 - 2.1 [The Core Idea](#)
 - 2.2 [What the System Does](#)
 - 2.3 [What Changes for the Engineer](#)
 3. [POC Architecture](#)
 - 3.1 [Architectural Approach](#)
 - 3.2 [System Layers](#)
 - 3.3 [The Pipeline Flow](#)
 - 3.4 [The Four Agents](#)
 - 3.5 [Database Schema \(PostgreSQL\)](#)
 4. [Architecture Flow — How It Works](#)
 - 4.1 [Step-by-Step Flow](#)
 - 4.2 [The Idempotency Guarantee](#)
 - 4.3 [The Live Routing Change](#)
 - 4.4 [Persona-Specific Views](#)
 5. [Technology Stack](#)
 6. [Execution Timeline — 6 Weeks](#)
 7. [Future Scope — Post-POC / Production Grade](#)
 8. [Full Production Architecture](#)
 - 8.1 [All Layers](#)
 - 8.2 [Why This Architecture Is Structured This Way](#)
 9. [Final POC Demo](#)
 10. [Summary](#)
-

1. Problem Statement

Context: HPE manages hybrid cloud solutions across servers, storage, networking, and software. Each product area is owned by a different engineering team, each using a different bug tracking system. When a customer reports an issue, that issue may span multiple components simultaneously — and the systems tracking those components are completely disconnected.

1.1 The Core Problem

When a customer reports a single issue — for example, "unable to create a VM" — the root cause can span multiple HPE components simultaneously. The current manual process requires engineers to:

- Read and interpret each bug report manually across disconnected systems.
- Identify which components are affected — with no AI assistance.
- Duplicate and copy-paste tickets between JIRA, Bugzilla, and GitHub — causing context loss.
- Track resolution across each team's separate tracker with no consolidated view.
- Manually piece together a status update for the customer from multiple systems.

1.2 The Four Fragmentation Problems

Problem	Impact
Scattered data	Engineers must search across JIRA Cloud, JIRA On-Prem, and Bugzilla separately to understand the state of a single customer problem. There is no single pane of glass.
No Unified View	Customers and executives have no single status page. Each team gives separate, inconsistent updates.
Manual triage & routing	Incoming bugs often end up in the wrong repository. Engineers manually reclassify and re-file them. This wastes time and increases error rates.
Lost lineage & context	When a bug is copied or reclassified into another system, the relationship between the original and the new ticket is not maintained, leading to inconsistent tracking.

1.3 Example Scenario

Customer reports: "I cannot create a VM in our private cloud." The failure could be caused by storage (disk quota), server (hypervisor timeout), or networking (vSwitch failure) — or all three simultaneously. Today: 3 separate engineers manually file 3 separate tickets with no linkage. Resolution time: 20–30 minutes per bug. With our system: fully automated in under 10 seconds.

2. Proposed Solution

2.1 The Core Idea

Build an agentic, multi-repository bug management system that acts as a single unified interface for all bug-related workflows — create, read, update, delete — irrespective of where the bug actually lives.

One customer submission → AI identifies all affected components → tickets created in all relevant trackers simultaneously → one unified view shows every ticket's status across all systems.

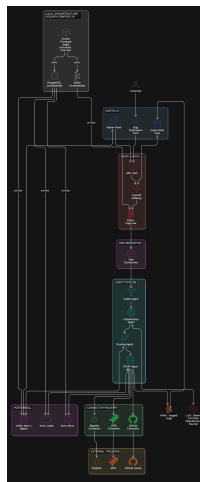
2.2 What the System Does

- **Multi-repository write:** One bug submission automatically creates tickets in JIRA Cloud, JIRA On-Prem, and Bugzilla simultaneously based on which components are affected.
- **Consolidated unified view:** One screen shows the status of all tickets for one customer problem. The customer sees a plain-English ETA, not three separate ticket numbers.
- **Federated search:** Engineers search from one place. Results come from all trackers merged and ranked. No context switching.
- **Automated classification and routing:** The LLM reads the bug description, identifies which HPE components are affected, and routes to the correct teams automatically using a configurable routing table.
- **Lineage preservation:** When a bug is copied or routed to multiple systems, the relationship is stored. Every ticket links back to the original customer case.

2.3 What Changes for the Engineer

State	Experience
Before	Manually read bug, guess component, open 3 browsers, copy-paste tickets, track status separately — slow and inefficient..
After	Bug submitted once. AI classifies, routes, creates all tickets, and sends team alerts — quick. Engineer only sees their relevant ticket.

3. POC Architecture



3.1 Architectural Approach

The POC uses a lean, focused architecture that proves the core value proposition. The full production architecture is designed and ready — the POC deliberately keeps the surface area small so every component can be demonstrated cleanly.

HYBRID DESIGN - Kafka at the entry boundary handles traffic spikes (always available). The Task Orchestrator enforces strict sequential agent execution inside (deterministic). Neither pattern alone achieves both goals — the combination gives us zero data loss + accurate results + enterprise scale.

3.2 System Layers

Layer	Components	What it does
Entry	Unified UI + FastAPI gateway + JWT auth	Customer submits one bug. System validates, authenticates, and publishes to Kafka.
Queue	Kafka	Submission queue. Decouples entry from processing. At-least-once delivery guaranteed.
Orchestration	Task orchestrator	Coordinates 4 agents in sequence. Retry, circuit breaker, idempotency. Saves state to PostgreSQL after every step.
Agents	Intake, Classification, Routing, CRUD	The intelligence pipeline. Each agent has single responsibility. Classification calls LLM API.
Connectors	JIRA Cloud, JIRA On-Prem (mock), Bugzilla	Adapter pattern. All implement same interface. Adding a new tracker = one new file.
Data store	PostgreSQL only	cases, work_items, cmdb_team_registry, outbox. All state lives here.
External	JIRA Cloud, Bugzilla instances	Real tickets created here. Source of truth for ticket status.

3.3 The Pipeline Flow

Customer fills form → FastAPI receives → Kafka queues → Intake normalises → LLM classifies + identifies components → Routing looks up CMDbTeamRegistry → CRUD creates tickets in parallel (JIRA + Bugzilla) → Unified UI shows all tickets for one case → Engineer searches across all systems from one box

3.4 The Four Agents

Agent	Type	Responsibility
1. Intake	Pure Python	Normalise text, generate case_id, extract keywords. No LLM. Cannot fail.
2. Classification	LLM	Returns severity (P0-P3) AND components[] affected. Retry 3x. Keyword fallback if LLM fails.
3. Routing	DB lookup	For each component: SELECT from cmdb_team_registry, get team + tracker + project. Zero LLM. Pure database query.
4. CRUD	API calls	Create tickets in all trackers in parallel. Outbox idempotency check before each creation — no duplicates.

3.5 Database Schema (PostgreSQL)

Table	Key fields	Purpose
cases	case_id, summary, severity, status, components[], source	One row per customer problem. The root entity everything references.
work_items	case_id FK, team, tracker, project, external_id, url, status	One row per ticket per team. This is the unified view data. JOIN with cases = single pane of glass.
cmdb_team_registry	component, team, tracker, project, active	Routing configuration. Change one row = routing changes instantly. No code change, no restart.
outbox	idempotency_key (SHA256), payload, status	Prevents duplicate tickets on crash + retry. Non-negotiable reliability mechanism.

Key insight: the JOIN that powers the unified view is simply: `SELECT c.*, w.team, w.tracker, w.external_id, w.url FROM cases c JOIN work_items w ON c.case_id = w.case_id WHERE c.case_id = ?` — one query, all trackers, one screen.

4. Architecture Flow — How It Works

4.1 Step-by-Step Flow

Step	Actor	What happens
1	Customer / UI	Submits bug form with summary and description. No severity selection — the system decides that.
2	FastAPI gateway	Validates request. Checks JWT role. Publishes event to Kafka. Returns 202 Accepted immediately — the caller never waits for processing.
3	Kafka	Holds the submission safely. Orchestrator consumes at its own pace. At-least-once delivery — no submission is ever lost.
4	Intake Agent	Normalises text, removes noise, generates case_id (UUID), extracts technical keywords. Writes to cases table with status='received'.
5	Classification Agent	Sends bug description to LLM API. LLM returns severity (P0–P3) AND components[] (e.g. ['StorageController', 'ComputeNode']). Updates cases table with severity and components.
6	Routing Agent	For each component in components[]: queries cmdb_team_registry to find team, tracker, and project. Creates one work_item row per team with external_id=NULL.
7	CRUD Agent	For each work_item: checks outbox table (prevents duplicates). Calls the correct connector in parallel. Updates work_items with real ticket IDs and URLs.
8	Unified UI	Case detail page shows all tickets, all teams, all statuses in one view. Customer sees plain English ETA. Engineer sees clickable ticket links + LLM reasoning.

4.2 The Idempotency Guarantee

The CRUD Agent implements the Outbox pattern. Before creating any ticket, it computes `SHA256(case_id + team + tracker)` and checks the outbox table. If the record exists, it returns the stored ticket ID. If not, it creates the ticket then writes to the outbox. This means the agent can crash and restart at any point without creating duplicate tickets.

If the system crashes between creating STO-108 and recording it: on restart, the outbox check finds nothing, creates STO-108 again — but this is the same ticket because the Outbox prevents the creation, not the crash. Result: zero duplicates, always.

4.3 The Live Routing Change

The CMDbTeamRegistry is a database table, not code. When a team switches trackers:

- One SQL UPDATE changes NetworkTeam tracker from 'bugzilla' to 'jira'
- Zero code changes. Zero service restarts.
- The very next bug submitted is routed to JIRA for NetworkTeam.
- This is the single most powerful demo moment — live configuration that changes system behaviour instantly.

4.4 Persona-Specific Views

Persona	What they see	What they don't see
Customer	Case status in plain English. ETA. 'Being investigated by 2 teams, expected resolution in 48 hours.'	Ticket numbers, tracker names, engineering details, LLM reasoning.
Engineer	All ticket links (STO-108, BZ-302). LLM classification reasoning. Which components were identified & why.	Executive SLA metrics, internal team assignments they don't own.
Executive	Aggregate dashboard: open P0s, P1s, average resolution time, SLA compliance.	Individual ticket details, raw LLM output, engineering view.

5. Technology Stack

Category	Technology	Why this choice
Backend	FastAPI (Python)	Async-first, Pydantic validation, automatic API docs.
Queue	Apache Kafka	At-least-once delivery. Submission buffer. Decouples entry from processing.
Database	PostgreSQL	ACID guarantees for case data, routing config, and idempotency keys. CP — consistency is non-negotiable.
LLM	Groq (Llama 3) / Gemini	Open-source LLM; pluggable — swap with one config change
Frontend	React/ Streamlit	Component-based. The persona views (customer/engineer/exec) are clean component switches.
Auth	JWT (mock users in POC)	Role-based access: customer, engineer, executive. Azure AD SSO in production.
Containerisation	Docker Compose	One command starts everything. <code>docker-compose up</code> — postgres + backend + frontend + kafka.
Connector pattern	Python ABC (Adapter)	BaseConnector interface. All trackers implement create/get/search. Adding new tracker = one file.
K8s (future)	Kubernetes + HPA	Manifests written and ready to show. HPA scales orchestrator 1→10 pods on queue depth.

6. Execution Timeline — 6 Weeks

Week	Focus	Deliverables	Exit criteria
Week 1	Infrastructure + entry layer	Docker Compose: postgres + backend + frontend + kafka; PostgreSQL schema (all 4 tables + seed CMDb); FastAPI gateway: POST /bugs, GET /bugs/{id}, GET /search; JWT auth with 3 roles	POST /bugs returns 202. Row in DB.
Week 2	Orchestrator + Intake + Classification	Task orchestrator: state machine, retry, circuit breaker; Intake Agent: normalise, UUID, keywords; Classification Agent: LLM API + retry + fallback	Bug classified with severity + components in DB.

Week	Focus	Deliverables	Exit criteria
Week 3	Routing + CRUD + Connectors	Routing Agent: CMDB lookup, hard policies; CRUD Agent: Outbox idempotency; JIRA Cloud connector (real REST API); Bugzilla connector (mock)	Real JIRA ticket created. Outbox prevents duplicate on retry.
Week 4	Unified UI	Submit bug page with real-time classification result; Case detail page: all tickets, all statuses, persona views; Dashboard: list all cases, filter by severity/status	End-to-end: form to unified view works.
Week 5	Federated search + admin panel	Federated search: parallel calls to JIRA + Bugzilla, merged results; Admin panel: live CMDB edit (the routing change demo)	Search returns results from all systems. Live routing change works.
Week 6	Polish + demo prep	Edge cases: JIRA down, LLM slow, bad input handled; Docker Compose verified on clean machine;\	Full demo.

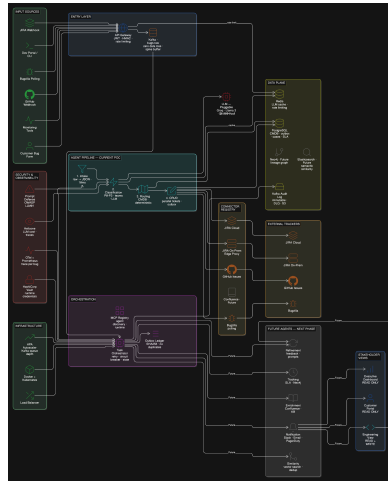
7. Future Scope — Post-POC / Production Grade

The POC proves the core loop. Everything below is designed, documented, and architecturally ready. Each item has a clear trigger for when it becomes necessary.

Component	When to add	What it enables
Redis cache and Rate Limiting	When monitoring tools fire the same alert 100+ times/day	Classification results cached 5 min. LLM cost drops 90%. Same bug, instant response.
Enrichment Agent	After core pipeline is stable	LLM receives KB articles alongside bug description. Classification accuracy improves.
Similarity / dedup Agent	After adding production data	Finds semantically similar past bugs. Monitoring alert deduplication — 500 alerts → 1 ticket.
Neo4j lineage graph	When multi-hop relationship traversal is needed	Complete audit trail. Relationship based queries in milliseconds.
Elasticsearch	When semantic search across historical data is needed	Meaning-based search. 'Find bugs like this one' without shared keywords.
SLA Agent	Before going to production customer traffic	Automated deadline tracking.
Refinement Agent	After 2+ months of correction data	Pattern detection. Auto-updates routing rules when engineers correct the same mistake 5+ times.
OpenTelemetry + Grafana	When you have 5+ services to debug across	Distributed traces. Find any failure across all agents in under 30 seconds.
HashiCorp Vault	Before connecting to production systems	All credentials runtime-fetched. Auto-expiring leases. Zero hardcoded secrets.
Kubernetes + HPA	When Docker Compose is the bottleneck	Orchestrator scales 1→10 pods under load. Zero-downtime deployments. HPA manifests ready now.
Azure AD SSO	When connecting to identity system	Real OIDC login. JWT tokens issued by

Component	When to add	What it enables
		identity provider.
MCP protocol	When 10+ external systems connect dynamically	Runtime tool discovery. New trackers register without restarting orchestrator.
Arize Phoenix / Helicone	When LLM observability is required	Prompt traces, token cost tracking, classification quality monitoring per severity.

8. Full Production Architecture



8.1 All Layers

Layer	Components
Entry	Customer Bug Form · Developer Portal / CLI · Monitoring Tools (Prometheus, Datadog webhooks) · Unified API Gateway (FastAPI, REST, GraphQL, Webhooks) · Azure AD SSO (OIDC, JWT, RBAC)
Queue	Apache Kafka — submission buffer, audit log, 30-day retention, S3 archive after 30 days, 7-year compliance retention
Orchestration	Task Orchestrator (retry, circuit breaker, loop detector, token budget) · MCP Server + Agent Registry (runtime tool discovery) · Outbox / Idempotency Ledger (SHA256 keys)
Agents (10)	1. Intake · 2. Enrichment (Confluence) · 3. Classification · 4. Similarity · 5. Routing · 6. CRUD · 7. SLA · 8. Lineage · 9. Notification · 10. Refinement (feedback loop)
LLM backends	Claude Sonnet · GPT-4o · Llama/Groq (10x faster) · AWS Bedrock (Privacy) · Ollama (on-premise).
Connectors	Confluence (CQL) · JIRA Cloud (REST+HMAC) · JIRA On-Prem (Edge Proxy + Secure Tunnel) · Bugzilla (REST+polling) · GitHub Issues (REST+HMAC) · Custom MCP-backed repos
Data plane	Redis (cache, rate limit) · PostgreSQL (cases, SLA, outbox, CMDb) · Neo4j (lineage graph, SOC2 audit) · Kafka (immutable event log) · Elasticsearch (semantic search) · DLQ Store (failed events)
Security	HashiCorp Vault (runtime-only credentials) · HMAC webhook verification · Prompt injection defence (OWASP LLM01) · PII masking before LLM calls · TLS on all connections
Observability	OpenTelemetry (traces + spans) · Prometheus (metrics, SLA counters) · Grafana (dashboards, alerts) · Helicone (POC LLM observability) · Arize Phoenix

Layer	Components
	(production, on-premise)
Infrastructure	Docker Compose (dev) → Kubernetes (prod) · HPA Autoscaler (1→10 orchestrator pods on queue depth) · Docker Registry · CI/CD Pipeline (build, test, deploy)
Notifications	Slack (engineer alerts P0/P1) · SendGrid (customer email) · PagerDuty (P0 phone escalation) · MS Teams (team alerts) · WebSocket (real-time executive dashboard)
External trackers	JIRA Cloud · JIRA On-Prem · Bugzilla · GitHub Issues · Custom MCP-backed repositories (source of truth — tickets live here)

8.2 Why This Architecture Is Structured This Way

- **Hybrid Kafka + Orchestrator:** Kafka handles traffic spikes at the boundary (AP — always available). The orchestrator enforces strict sequential agent execution (CP — consistent). Neither pattern alone achieves both goals.
- **Stateless agents + PostgreSQL state:** Every agent is stateless. State lives in PostgreSQL. Any agent can crash and restart — the orchestrator reads the last status and continues from exactly that point. This is what makes the system crash-safe.
- **Adapter pattern for connectors:** Every connector implements the same BaseConnector interface. The CRUD agent never imports JiraCloudConnector directly — it asks the registry. Adding GitHub Issues = one new file, zero other changes.
- **CQRS for dashboards:** The executive dashboard does not query the operational PostgreSQL tables. It reads pre-aggregated summaries updated by Kafka consumers. This prevents analytics queries from slowing down the write pipeline.
- **CMDBTeamRegistry as configuration:** Routing is data, not code. When StorageTeam switches from JIRA to Bugzilla — one SQL UPDATE, zero deployment. This is the most important architectural decision for long-term maintainability.

9. Final POC Demo

The demo proves every architectural claim with live behaviour. Each moment corresponds to a specific design decision.

Moment	Action	What it proves
Multi-repo write	Submit 'VM creation fails'. Watch orchestrator logs run 4 agents. Show JIRA ticket STO-108 + Bugzilla BZ-302 created.	Core value: one submission → multiple trackers. No manual filing.
Unified view	Open case detail page. Show all tickets, all teams, all statuses in one screen. Switch persona views (customer vs engineer).	Single pane of glass. Customer sees ETA. Engineer sees ticket links + reasoning.
Live routing change	In admin panel: change NetworkTeam tracker from Bugzilla to JIRA. Submit network bug. Watch it go to JIRA instead.	Routing as configuration. One SQL update, zero code change, immediate effect.
Federated search	Search 'VM provisioning'. Results from JIRA + Bugzilla merged. Each result shows which system it came from.	Single search box across all systems. No context switching.

10. Summary

What the POC delivers

- One bug submission creates tickets in all affected trackers simultaneously
- AI identifies which teams own which components — no manual guessing
- One unified view shows every ticket's status across all trackers
- Federated search across all bug systems from one search box
- Live routing configuration — change trackers without touching code

What is designed and ready for production

- Full 10-agent pipeline with Neo4j lineage, Elasticsearch similarity, Refinement loop
 - Kubernetes + HPA autoscaling manifests written and demonstrable
 - Full security stack: Vault, HMAC, prompt injection defence, PII masking
 - Complete observability: OpenTelemetry, Grafana, Arize Phoenix
 - Multi-LLM backend support: Claude, GPT-4o, Bedrock, Ollama
-